Journal of Systems Architecture 167 (2025) 103508

Contents lists available at ScienceDirect

Journal of Systems Architecture

journal homepage: www.elsevier.com/locate/sysarc

Inference framework supporting parallel execution across heterogeneous accelerators

Philkyue Shin [a], Myungsun Kim [b], Seongsoo Hong [a,*]

[a] Department of Electrical and Computer Engineering, Seoul National University, Seoul 08826, Republic of Korea
[b] Department of Applied Artificial Intelligence, Hansung University, Seoul 02876, Republic of Korea

ARTICLE INFO

Keywords:
Deep learning
Inference
Heterogeneous accelerators
NPU
Scheduler

ABSTRACT

The growing demand for on-device deep learning inference, particularly in latency-sensitive applications, has driven the adoption of heterogeneous accelerators that incorporate GPUs, DSPs, and NPUs. While these accelerators offer improved energy efficiency, their heterogeneity introduces significant programming complexity due to reliance on vendor-specific APIs. Existing deep learning inference frameworks, such as LiteRT, provide high-level APIs and support multiple backend APIs. However, they lack the ability to exploit parallel execution across heterogeneous accelerators. This paper introduces a novel inference framework that overcomes this limitation. Our framework utilizes a batch inference API to enable parallel execution across heterogeneous accelerators. The framework schedules heterogeneous accelerators to process batched inputs concurrently. To address the challenge of integrating commercial NPU APIs that do not support LiteRT, we develop a portable hooking engine. Furthermore, the framework mitigates accuracy inconsistencies arising from diverse quantization methods by dynamically adjusting postprocessing parameters to balance accuracy and latency. The proposed framework minimizes both average turnaround time and postprocessing time. Experimental results on a Qualcomm SA8195 SoC with Mobilint and Hailo NPUs demonstrate significant performance improvements compared to existing inference frameworks.

1. Introduction

The deployment of deep learning (DL) models has shifted from cloud-centric to mobile-centric for on-device artificial intelligence [1–3]. This transition has enabled the development of various latency-sensitive applications, including advanced driver assistance systems (ADAS) [4], posture tracking on smartwatches [5], mobile virtual reality [6], and drone control [7].

To meet the complex computational demands of DL applications within a limited power budget, commercial hardware for mobile devices integrates a variety of accelerators, including GPUs, DSPs, and NPUs [2]. The adoption of multiple accelerators, rather than a single accelerator, has emerged as a strategy to enhance energy efficiency [46]. Each accelerator is designed for hardware acceleration of specific computations frequently used in modern applications [8–10].

However, these heterogeneous accelerators present a significant challenge to developers, requiring them to utilize multiple Application Programming Interfaces (APIs). While GPUs often offer a unified API option, such as CUDA [11] or OpenCL [11], DSPs and NPUs typically rely on proprietary APIs. Examples include the Hexagon API [12] for Qualcomm DSPs, the Maccel API [13] for Mobilint NPUs, the HailoRT API [14] for Hailo NPUs, and the NNAPI [15] for Android devices.

Current state-of-the-art DL inference frameworks, including LiteRT [16], ONNX Runtime [17], and ExecuTorch [18], provide developers with a high-level API while supporting multiple backend APIs to support the GPUs, DSPs, and NPUs. However, these frameworks are typically limited to executing a model on a single accelerator at a time. This restriction, often stemming from underlying backend library limitations (as will be detailed in Section 6), prevents the parallel utilization of multiple accelerators. Furthermore, once a primary accelerator is designated, these frameworks prioritize its use for executing the model as much as possible. Consequently, these frameworks fail to fully exploit the potential of parallel heterogeneous computing to improve latency in on-device DL inference.

To address this limitation, numerous researchers have explored dynamically partitioning models into subgraphs and assigning them to multiple accelerators at runtime [45–52]. However, this strategy is incompatible with most commercial heterogeneous accelerators.

* Corresponding author.
E-mail addresses: pkshin@redwood.snu.ac.kr (P. Shin), kmsjames@hansung.ac.kr (M. Kim), sshong@redwood.snu.ac.kr (S. Hong).

https://doi.org/10.1016/j.sysarc.2025.103508
Received 6 March 2025; Received in revised form 28 May 2025; Accepted 19 June 2025
Available online 20 June 2025
1383-7621/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Vendors mandate that developers convert models into proprietary representations during a pre-runtime phase; these formats are optimized for the vendors' specific DSPs and NPUs. Consequently, because each accelerator requires a distinct model representation, dynamically decomposing a model into subgraphs at runtime for execution across heterogeneous accelerators is practically infeasible.

This paper aims to develop an inference framework that enables concurrent execution across heterogeneous accelerators. To achieve this, we leverage a batch inference API, a feature supported by contemporary DL inference frameworks. Batch execution of multiple inputs has been introduced to reduce inference time compared to repeated single-input processing. Frameworks such as LiteRT, ONNX Runtime, and ExecuTorch all support this capability. A relevant example highlighting the need for batch inference is surroundings recognition in ADAS, which requires vehicles to be equipped with multiple sensors [19]. Fig. 1 depicts a vehicle that motivated this research, designed for ADAS by the Korea Automotive Technology Institute [20]. Equipped with eight cameras and four LiDAR sensors, this vehicle underscores the need for concurrent processing of multiple inputs. In such scenarios, batch inference becomes a crucial capability.

The proposed inference framework leverages heterogeneous accelerators to process batched inputs concurrently, aiming to reduce the latency of DL applications. This framework comprises three key components: a hooking engine, an accelerator scheduler, and a postprocessing parameter adjuster. The hooking engine allows the LiteRT API, a representative DL inference framework API, to utilize commercial NPUs on the target system. These NPUs, which employ proprietary APIs, are unsupported by contemporary DL inference frameworks. Notably, current inference frameworks offer limited support for commercial NPUs, often supporting only a select few. The hooking engine is designed for high portability, requiring no modifications to the LiteRT library itself. Instead, developers only need to include our header file and link the provided library—no alterations to their existing LiteRT code are necessary.

The accelerator scheduler determines which accelerator to process inputs in a batch. The idea of the scheduler is to assign each input to the accelerator that will have the shortest turnaround time. The effectiveness of the proposed scheduling algorithm is demonstrated by mathematically proving that the average turnaround time is minimized. We also prove that the algorithm guarantees the first-come, first-finish. In addition, the algorithm bounds the maximum turnaround time to a constant value.

The postprocessing parameter adjuster aims to minimize postprocessing latency, which is particularly significant for compact models. For example, the postprocessing latency of EfficientDet Lite 0 accounts for 80 % of the total DL application latency. To mitigate this, we propose a strategy that selectively compromises the accuracy of specific accelerators. Heterogeneous accelerators exhibit varying accuracies even when executing the same model, due to the use of different quantization techniques. For instance, the Mobilint NPU employs percentile and KL divergence quantization, while the Hailo NPU utilizes its proprietary IBC and QFT quantization methods [34,35], and LiteRT uses its own quantization approach [36]. Consequently, the mean average precision (mAP) for the YOLOv8 medium model varies: 0.476 for the Mobilint NPU, 0.484 for the Hailo NPU, and 0.432 for LiteRT.

This variation in accuracy forces developers to accept the lowest accuracy among the heterogeneous accelerators when selecting a model to meet an application's accuracy requirements. Inspired by this, the postprocessing parameter adjuster dynamically modifies parameters to reduce postprocessing latency when processing outputs from accelerators with higher accuracy than the lowest accuracy.

We evaluated our inference framework on the target hardware platform, comprising the Qualcomm SA8195 SoC, the Mobilint NPU, and the Hailo NPUs. A comparative analysis against the proprietary inference frameworks provided by the NPU vendors and the open-source inference framework LiteRT demonstrated a significant reduction in application latency, reaching up to 91 %.

Our contributions are summarized as follows:

- We introduce an inference framework that enables parallel execution across commercial heterogeneous accelerators via LiteRT.
- We present a provably optimal scheduling algorithm that minimizes average turnaround time while guaranteeing first-come, first-finished processing and bounding the maximum turnaround time. We formally prove the properties of the scheduling algorithm.
- We propose a method to minimize postprocessing latency by selectively sacrificing the accuracy of accelerators with higher accuracy than others.

The rest of the paper is organized as follows. Section 2 briefly introduces LiteRT. Section 3 describes the problem at hand. Section 4 presents our inference framework. Section 5 reports the experimental evaluation. Section 6 reviews studies on DL inference frameworks. Finally, Section 7 concludes the paper.

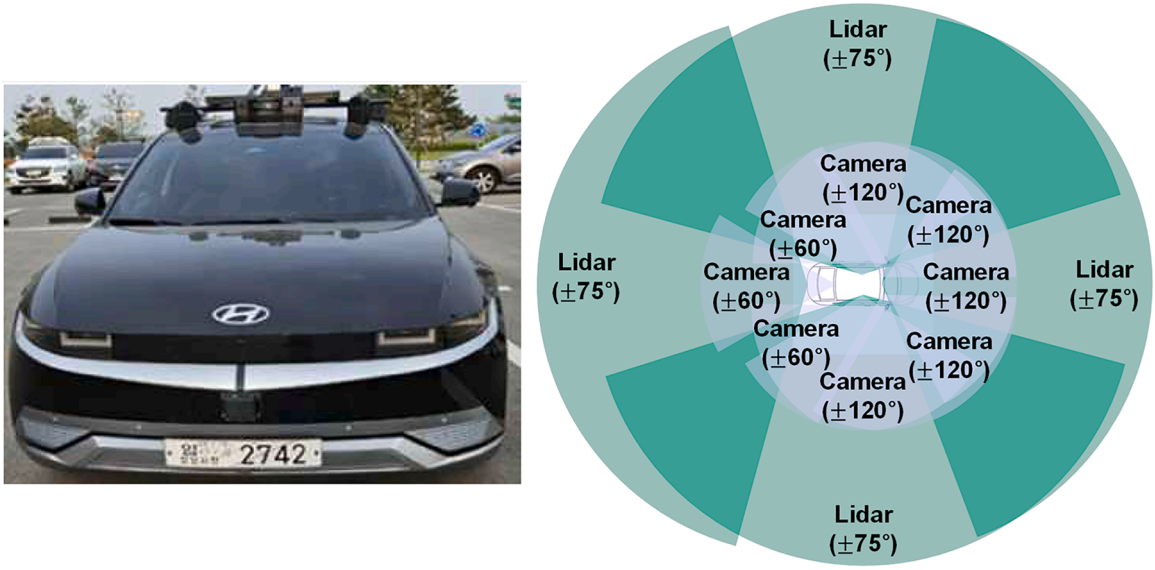


Fig. 1. Vehicle equipped with multiple cameras and lidars for ADAS.

2. Background

LiteRT, formerly known as TensorFlow Lite, is a prominent inference framework for embedded systems, released by Google in 2017 [16]. Fig. 2 illustrates the LiteRT workflow. LiteRT supports models trained using various deep learning frameworks. To convert a trained model to a LiteRT model, developers must use the LiteRT converter at pre-runtime. During the conversion, it is essential to quantize the model to ensure its compatibility with DSPs and NPUs. The quantization scheme employed by the converter determines the accuracy of the converted model. The converted model is then transferred to the target embedded devices, where it is run via the LiteRT API.

The LiteRT interpreter reads the model file and executes it on CPUs. To utilize a GPU, DSP, or NPU, developers must employ the appropriate LiteRT delegate. The GPU delegate executes the model on the GPU, with the CPU serving as a fallback if certain layers are unsupported by the GPU. The Hexagon delegate executes the model on Qualcomm Hexagon DSPs, while the NNAPI delegate leverages the Android NNAPI for model execution. These delegates rely on the OpenCL, OpenGL ES, Hexagon, and NNAPI libraries, respectively.

The execution model of LiteRT is synchronous, meaning that the execution of a model occurs in a single, uninterrupted sequence. Initiation of a subsequent model execution is only permitted after the completion of the preceding execution. This synchronous nature restricts LiteRT to executing a single model at a time within a task. While LiteRT does offer batch execution capabilities for concurrently processing multiple inputs, this feature is currently limited to its CPU backend. Support for batch inference is not available on its GPU and Hexagon delegates.

3. Problem description

This paper aims to develop an inference framework that supports parallel execution across heterogeneous accelerators via LiteRT. To formally define this objective, we first describe the target hardware and application, followed by a presentation of the problem statement.

3.1. Target system

Our target hardware is an embedded system that possesses heterogeneous accelerators. To exemplify this, we selected a board specifically designed for this research, as shown in Fig. 3. The board's main System-on-Chip (SoC) is a Qualcomm SA8195, featuring eight CPU cores, a GPU, and a DSP. Two variants of the board were used: one equipped with a Mobilint Aries NPU connected to the SoC via PCIe, and the other with three Hailo-8 NPUs. Both board variants include Fakra connectors and Automotive Ethernet for connecting multiple sensors.

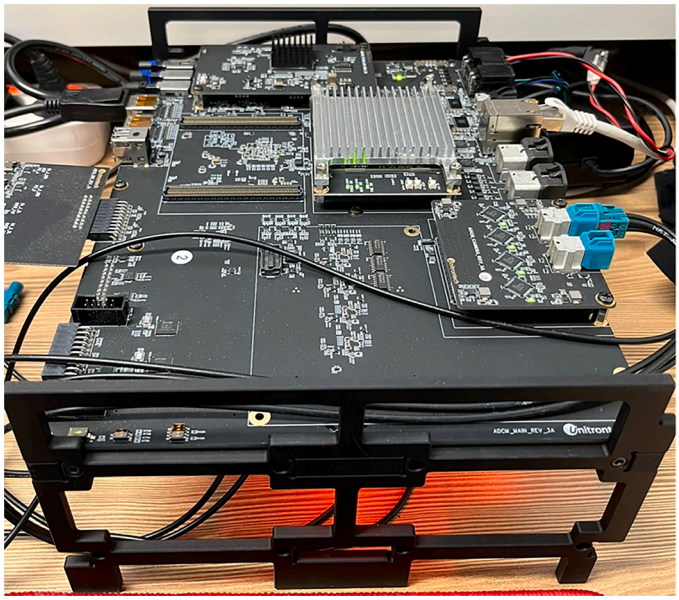


Fig. 3. Target hardware.

The target application is a batch inference application illustrated in Fig. 4. The timeline for the batch inference application is described as follows. First, the application collects sensor data from multiple sensors. Second, it preprocesses the collected sensor data to generate a batch input on CPUs. Third, it executes a model with the batch input on the accelerators. Until the batch inference is finished, preprocess stops. Fourth, as an output is generated from the accelerator, the output is postprocessed to produce the result on CPUs.

3.2. Problem definition

In order to run batch inference on heterogeneous accelerators, a multi-queue, multi-accelerator scheduling model can be considered, as

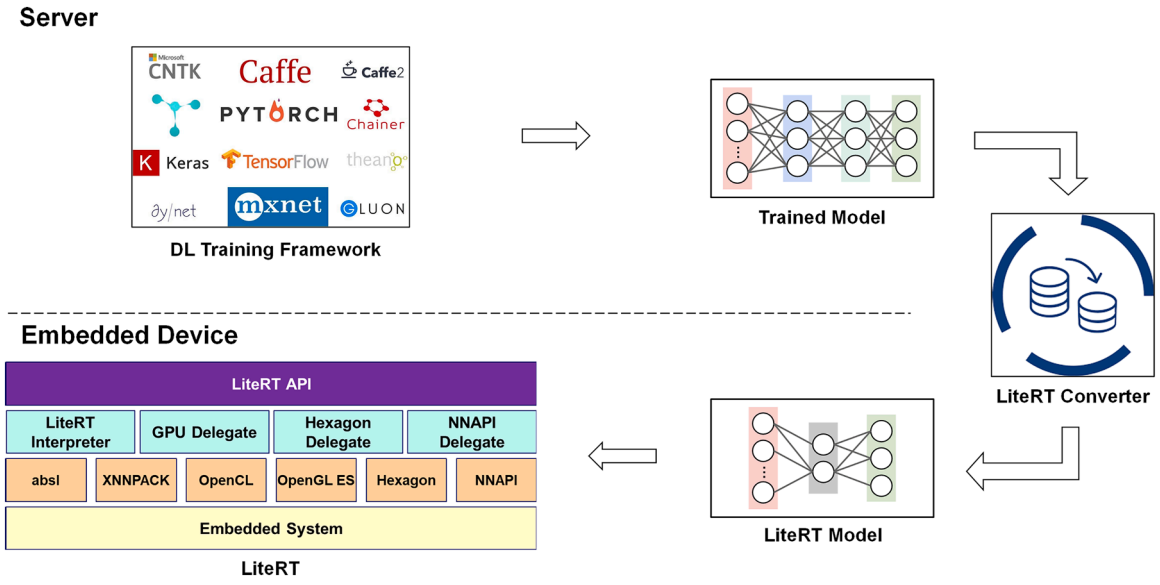


Fig. 2. Workflow for LiteRT.

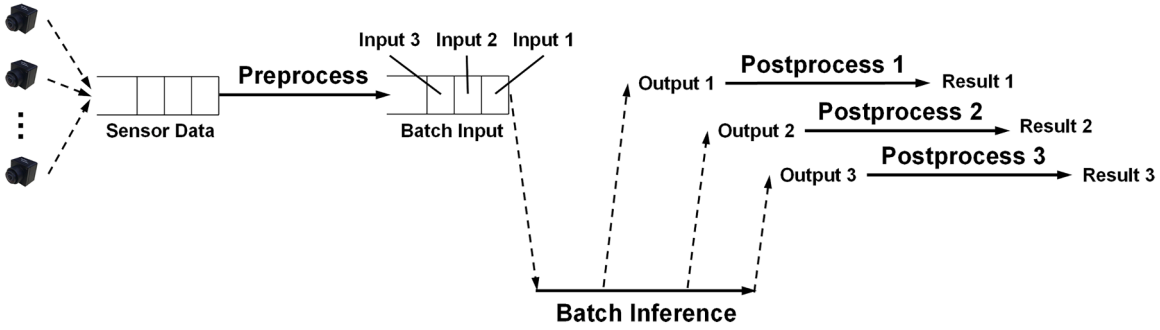


Fig. 4. Target application.

illustrated in Fig. 5. The batch input is provided as a queue, and the scheduler determines the assignment of each input to an accelerator queue. Each accelerator processes the inputs in its queue sequentially and generates outputs.

According to this model, the end-to-end latency from sensor data to final result is comprised of three distinct components, as illustrated in Fig. 6. First, the sensor data undergoes preprocess and becomes one of the inputs in batch input. Second, the input awaits execution in the accelerator queue, where it remains until the previous inputs in the queue have been processed. The input is then subjected to inference, yielding an output. Finally, this output undergoes postprocess, resulting in a final outcome.

In order to minimize the end-to-end latency, it is necessary to reduce preprocessing time, turnaround time, and postprocessing time depicted in Fig. 6. Among these latencies, our focus is on reducing the turnaround time and the postprocessing time, given that the preprocessing time is relatively short.

To formally define the turnaround time and the postprocessing time, we introduce the notations summarized in Table 1. These notations are defined within the scope of a single batch inference; they are reinitialized for each new batch inference.

We define τ_i as a job to execute the model on the ith input. Let the number of inputs in an input batch be n. Then, we need to schedule τ_1, τ_2, …, τ_n to the accelerators. Fig. 5 will help to understand the notations. Define q_j as the queue of the jth accelerator. Let the number of accelerators be m. When τ_i is scheduled to q_j, the number of inputs in q_j increases. We denote the number of jobs scheduled to q_j before τ_i as $k_{j,i}$. The turnaround time of τ_i is the sum of the wait time in the accelerator queue and the time required to execute τ_i. Since heterogeneous accelerators have different performance characteristics, the time needed to execute a model is different as well. We define the time needed to execute a model on q_j as c_j. We then define t_i as the turnaround time of τ_i. Obviously, t_i is $k_{j,i} \cdot c_j + c_j$.

We subsequently define o_i as the output of τ_i. The number of total outputs is equivalent to n. The postprocessing time of o_i is denoted as p_i. o_i becomes r_i after postprocess. Given the discrepancy in accuracy across different accelerators, the accuracy of r_i will be different. We define a_i as the accuracy of r_i.

Our goal is to minimize the arithmetic mean of t_i and the arithmetic mean of p_i, which are $1/n \cdot \sum_{i=1}^{n} t_i$ and $1/n \cdot \sum_{i=1}^{n} p_i$, respectively. The constraint is to avoid decreasing the minimum accuracy, represented as $min(a_1, a_2, \ldots, a_n)$.

4. Inference framework

The proposed inference framework operates in two primary steps to achieve the objective defined in the preceding section. Fig. 7 illustrates these operations of the framework. First, the accelerator scheduler determines the assignment of each input to an accelerator queue, aiming to minimize the arithmetic mean of t_i. The inference thread in the target application initiates the scheduler via LiteRT's batch inference API. The hooking engine intercepts this API call and invokes the scheduler. After the scheduler assigns inputs to the accelerator queues, it forks the worker threads. These worker threads obtain the inputs in each accelerator queue and execute them on the accelerators via the compatible APIs.

Second, the postprocessing parameter adjuster provides the postprocessing parameters that minimize the arithmetic mean of p_i while maintaining the minimum accuracy. Upon completion of the assignment of inputs, the accelerator scheduler furnishes the mapping relationship between inputs and their assigned accelerators to the postprocessing parameter adjuster. The postprocessing parameter adjuster possesses a set of tuples of score threshold and mAP for each accelerator that is provided offline, enabling it to ascertain the optimal score threshold for each output. These determined score thresholds are then provided to the postprocessing threads in the target application, where postprocess is performed on each output using the score threshold.

The proposed inference framework operates within the layered

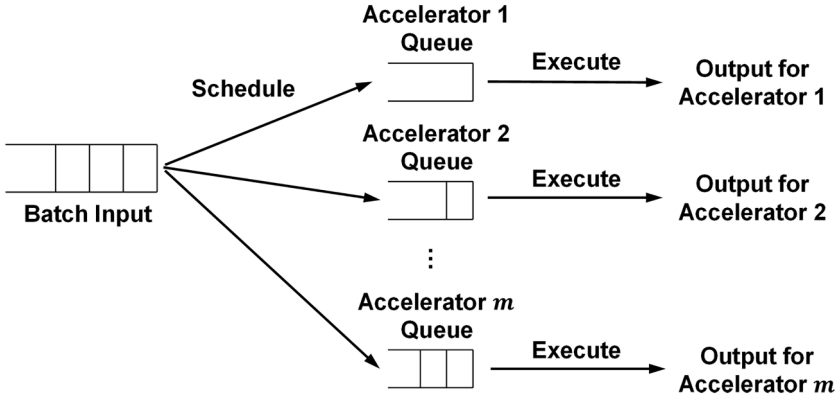


Fig. 5. Scheduling model for batch inference using heterogeneous accelerators.

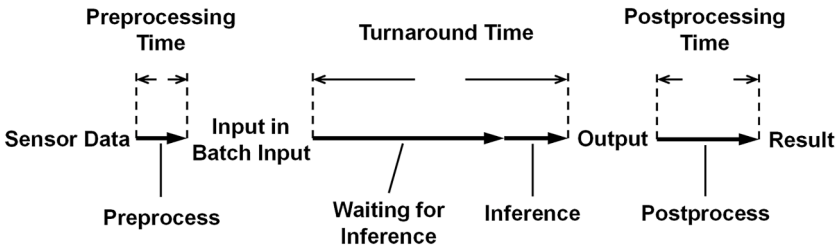


Fig. 6. End-to-end latency from sensor data to final result.

Table 1
Basic notations.

Symbol	Definition
τ_i	Job to execute model on the ith input
n	Number of inputs in an input batch
q_j	Queue of the jth accelerator
m	Number of accelerators
$k_{j,i}$	Number of jobs scheduled to q_j before τ_i
c_j	Time needed to execute a model on q_j
t_i	Turnaround time of τ_i
o_i	Output of τ_i
p_i	Postprocessing time of o_i
r_i	Result of o_i
a_i	Accuracy of r_i

architecture depicted in Fig. 8. The framework is built on top of an automotive-grade Linux operating system, and all necessary libraries and drivers, including LiteRT and its delegates, have been ported to ensure seamless integration. Crucially, the framework is designed for hardware independence. This is achieved primarily through the LiteRT, which support a wide range of hardware platforms via XNNPack, OpenCL, OpenGL ES, Hexagon, and NNAPI libraries. Furthermore, we provide a hooking engine to facilitate support for hardware utilizing proprietary libraries. The accelerator scheduler and postprocessing parameter adjuster components are also hardware-agnostic. The complete source code and a comprehensive porting guide are available in our GitHub repository [21].

The following sections detail the hooking engine, the accelerator scheduler, and the postprocessing parameter adjuster, respectively.

4.1. Hooking engine

The purpose of the hooking engine is to enable the LiteRT API to utilize Mobilint and Hailo NPUs. These NPUs rely on their proprietary Maccel and HailoRT libraries, which precludes their compatibility with the OpenCL, OpenGL ES, Hexagon, and NNAPI libraries used by LiteRT delegates. Consequently, the hooking engine has been developed to facilitate the LiteRT API to call the Maccel and HailoRT libraries.

Fig. 9 depicts the operation of the hooking engine. The hooking engine implements custom LiteRT APIs that have the same names and arguments as the originals. When the target application calls a LiteRT API, it is redirected to the corresponding custom LiteRT API. This custom API then invokes either the original LiteRT API or the appropriate Maccel/HailoRT API. Critically, the hooking engine does not require any

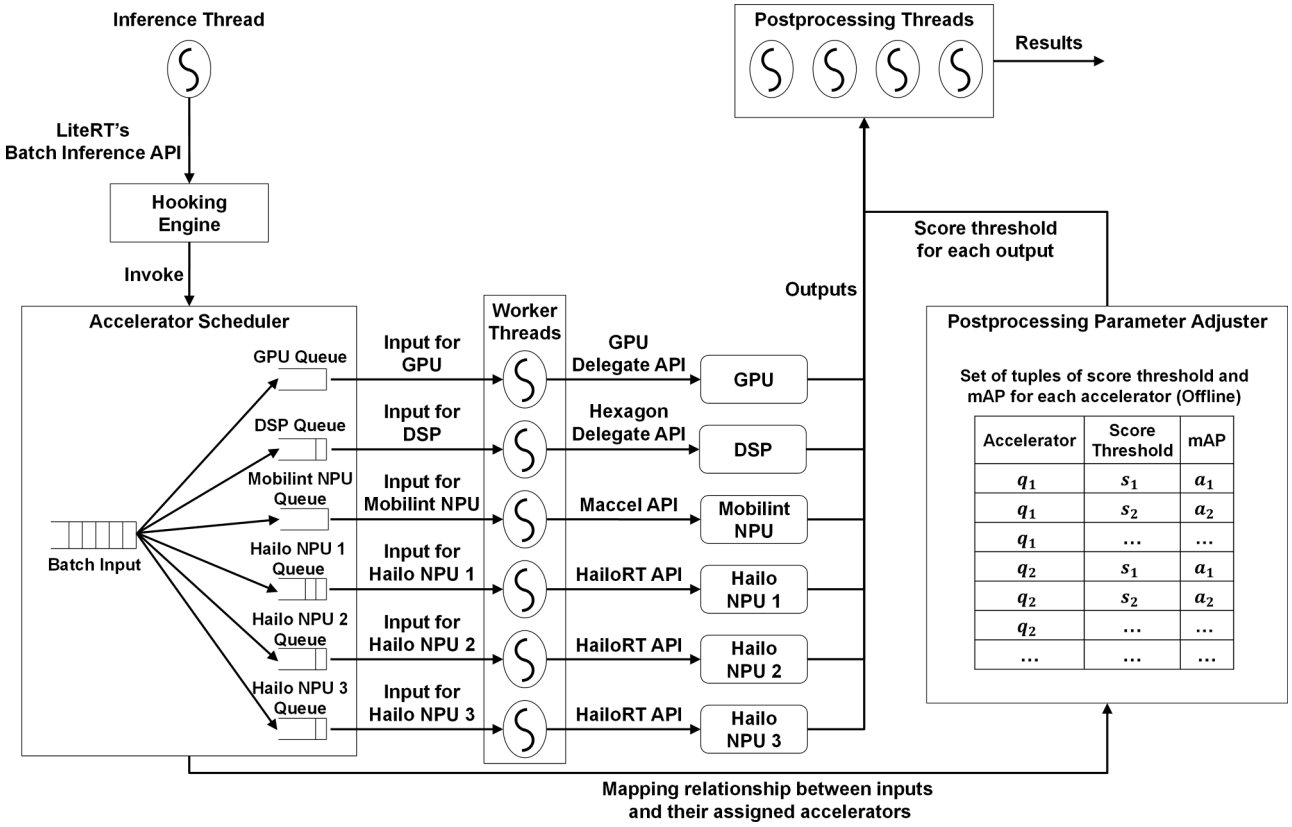


Fig. 7. Overview of the proposed inference framework.

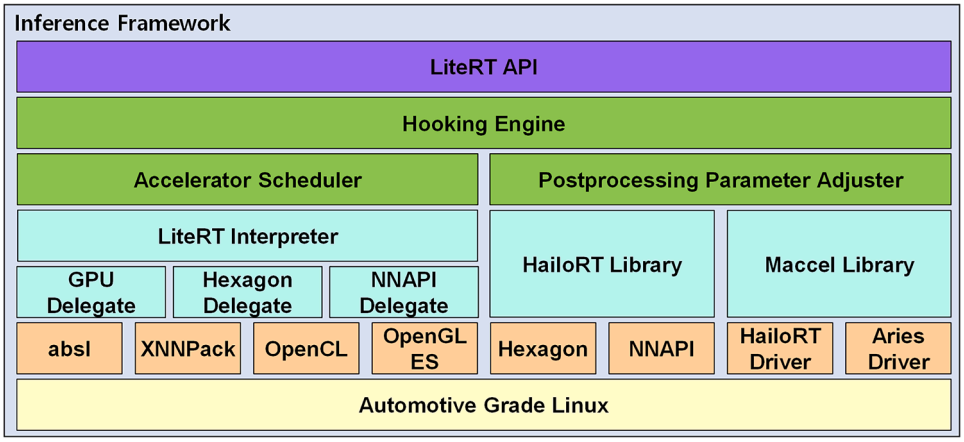


Fig. 8. Layered architecture of the proposed inference framework.

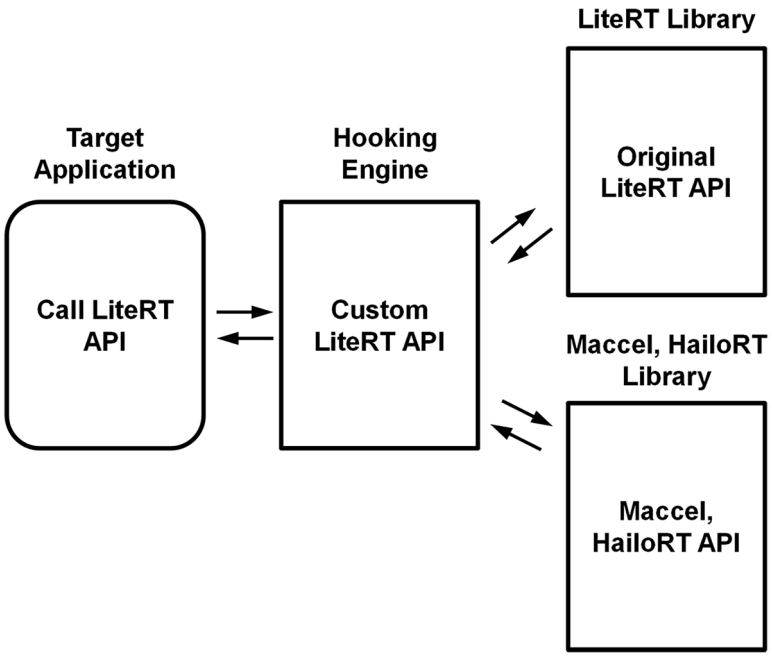


Fig. 9. Operation of hooking engine.

modification to the LiteRT library itself, ensuring compatibility with any version of the library.

The primary challenge in developing the hooking engine is invoking the original LiteRT API within the custom LiteRT API. The LiteRT C++ API includes classes and functions. To call a C++ function without using its name directly, one must know its address. However, the C++ standard prohibits taking the address of a constructor or destructor [22]. Consequently, the LiteRT C++ functions (specifically constructors and destructors) must be called directly within the custom functions of the same name.

In order to address this challenge, we make custom classes with the same names as the original LiteRT classes, but within a different namespace, and inheriting the original LiteRT classes. This inheritance allows us to call directly all original member functions and variables. We then overload about 30 functions, including creation of an interpreter object from a model file and parsing of information from input and output tensors, to call the Maccel and HailoRT libraries when needed. The batch inference API is also overloaded to invoke the accelerator scheduler. Since the LiteRT API does not map to the Maccel and HailoRT API one by one, the implementation requires precise coding techniques.

Developers can utilize the hooking engine by simply including the provided header file and linking the library. The header file instructs the C++ preprocessor to convert the namespaces, a process that occurs transparently to the developer.

4.2. Accelerator scheduler

The idea of the accelerator scheduler is to put τ_i on q_j that t_i will be minimum. It calculates t_i candidates if τ_i is scheduled to each q_j and selects the minimum. The algorithm of the accelerator scheduler is depicted in Fig. 10. The inputs are n, m, and c_j for $1 \leq j \leq m$. The outputs are t_i for $1 \leq i \leq n$ and q_j for $1 \leq j \leq m$. Line 1~3 initializes $k_{j,i}$ as 0. Line 5~7 calculates t_i candidates. Line 8~14 finds the minimum t_i candidate. Line 15~16 increases $k_{j,i}$ and puts τ_i on q_j. Line 5~16 is iterated for all τ_i.

This greedy algorithm minimizes the arithmetic mean of turnaround time. We will prove it.

Theorem 1. *Our algorithm minimizes* $1/n \cdot \sum_{i=1}^{n} t_i$.

Proof.

For any turnaround time u,

```
Input: n; m; c[j] for 1 ≤ j ≤ m
Output: t[i] for 1 ≤ i ≤ n; q[j] for 1 ≤ j ≤ m
 1: for j = 1 to m do
 2:   k[j] = 0
 3: end for
 4: for i = 1 to n do
 5:   for j = 1 to m do
 6:     temp_t[j] = (k[j] + 1) * c[j]
 7:   end for
 8:   t[i] = ∞
 9:   for j = 1 to m do
10:     if t[i] > temp_t[j] then
11:       t[i] = temp_t[j]
12:       index = j
13:     end if
14:   end for
15:   k[index] += 1
16:   enqueue(q[index], i)
17: end for
```

Fig. 10. Algorithm of acclerator scheduler.

$u = s{\cdot}c_j$

where s is a positive integer and j is an integer that $1 \le j \le m$.

Sort all u by ascending order and let them $u_1 \le u_2 \le \ldots$.

We will prove that $t_i = u_i$ for $1 \le i \le n$.

Let $U_1 = \{1{\cdot}c_1,\ 2{\cdot}c_1\ ,\ 3{\cdot}c_1\ ,\ldots\}$.

Let $U_2 = \{1{\cdot}c_2,\ 2{\cdot}c_2\ ,\ 3{\cdot}c_2\ ,\ldots\}$.

…

Let $U_m = \{1{\cdot}c_m,\ 2{\cdot}c_m\ ,\ 3{\cdot}c_m\ ,\ldots\}$.

Obviously, u is an element of one of U_j.

We will prove that our proposed algorithm is equivalent to the procedure that generates the sequence $u_1, u_2, \ldots$ by iteratively finding the minimum among the current head elements (least elements) of sets $U_1, U_2, \ldots, U_m$, selecting it as the next u_i, and conceptually removing it from its source set (i.e., advancing to the next element in that set).

Consider the illustrative example in Fig. 11 with initial sets $U_1 = \{2,\ 4,\ 6,\ 8,\ 10,\ \ldots\}$, $U_2 = \{3,\ 6,\ 9,\ 12,\ 15,\ \ldots\}$, and $U_3 = \{4,\ 8,\ 12,\ 16,\ 20,\ldots\}$.

To find u_1, we identify the least elements of U_1, U_2, and U_3, which are 2, 3, and 4, respectively. The minimum is $min(2,\ 3,\ 4) = 2$. Thus, $u_1 = 2$.

Since u_1 came from U_1, we remove $u_1 = 2$ from U_1. To find u_2, we find the minimum among the least elements of new set: 4 (from U_1), 3 (from U_2), and 4 (from U_3). The minimum is $min(4,\ 3,\ 4) = 3$. Thus, $u_2 = 3$.

Since u_2 came from U_2, we also remove $u_2 = 3$ from U_2. To find u_3, we find the minimum among the least elements of new set: 4 (from U_1), 6 (from U_2), and 4 (from U_3). The minimum is $min(4,\ 6,\ 4) = 4$. Thus, $u_3 = 4$.

This process of iteratively finding u_i is equivalent to our algorithm. In general, the following holds.

Let the least element of set S as $lst(S)$.

$$\begin{aligned} u_1 &= min(lst(U_1), lst(U_2), \ldots, lst(U_m)) \\ &= min(c_1, c_2, \ldots, c_m) \\ &= t_1 \end{aligned}$$

Let $u_1 = c_\alpha$ that $1 \le \alpha \le m$.

$$\begin{aligned} u_2 &= \min(lst(U_1), lst(U_2), \ldots, lst(U_\alpha - \{c_\alpha\}), \ldots, lst(U_m)) \\ &= \min(c_1, c_2, \ldots,\ 2{\cdot}c_\alpha, \ldots, c_m) \end{aligned}$$

Since $t_1 = u_1 = c_\alpha$, our algorithm schedules τ_1 to q_α.

$\therefore k_{j,2} = 0$ for $1 \le j \le \alpha$ and $(\alpha + 1) \le j \le m$, and $k_{\alpha,2} = 1$.

$\therefore t_2 = \min(c_1, c_2, \ldots,\ 2{\cdot}c_\alpha, \ldots, c_m) = u_2$.

After removing u_1, u_2, …, u_{i-1} from U_1, U_2, …, U_m, let the least element of U_j as $s_{j,i}{\cdot}c_j$.

Let $u_i = \min(s_{1,i}{\cdot}c_1, s_{2,i}{\cdot}c_2, \ldots, s_{m,i}{\cdot}c_m) = s_{\alpha,i}{\cdot}c_\alpha$.

We will prove that if $s_{j,i} = k_{j,i} + 1$ for $1 \le j \le m$, then $s_{j,i+1} = k_{j,i+1} + 1$ for $1 \le j \le m$.

$u_{i+1} = \min(s_{1,i+1}{\cdot}c_1,\ s_{2,i+1}{\cdot}c_2,\ \ldots,\ s_{m,i+1}{\cdot}c_m) = \min(s_{1,i}{\cdot}c_1,\ s_{2,i}{\cdot}c_2,\ \ldots,\ (s_{\alpha,i} + 1){\cdot}c_\alpha, \ldots, s_{m,i}{\cdot}c_m)$

$\therefore s_{j,i+1} = s_{j,i}$ for $1 \le j \le \alpha$ and $(\alpha + 1) \le j \le m$, and $s_{\alpha,i+1} = s_{\alpha,i} + 1$.

$$\begin{aligned} t_i &= \min\big((k_{1,i} + 1){\cdot}c_1, (k_{2,i} + 1){\cdot}c_2, \ldots, (k_{m,i} + 1){\cdot}c_m\big) \\ &= \min\big(s_{1,i}{\cdot}c_1, s_{2,i}{\cdot}c_2, \ldots, s_{m,i}{\cdot}c_m\big) \\ &= s_{\alpha,i}{\cdot}c_\alpha \\ &= (k_{\alpha,i} + 1){\cdot}c_\alpha \end{aligned}$$

Therefore, our algorithm schedules τ_i to q_α.

$\therefore k_{j,i+1} = k_{j,i}$ for $1 \le j \le \alpha$ and $(\alpha + 1) \le j \le m$, and $k_{\alpha,i+1} = k_{\alpha,i} + 1$.

If $s_{j,i} = k_{j,i} + 1$ for $1 \le j \le m$, then $s_{j,i+1} = s_{j,i} = k_{j,i} + 1 = k_{j,i+1} + 1$ for $1 \le j \le \alpha$ and $(\alpha + 1) \le j \le m$, and $s_{\alpha,i+1} = s_{\alpha,i} + 1 = k_{\alpha,i} + 1 + 1 = k_{\alpha,i+1} + 1$.

$\therefore$ If $s_{j,i} = k_{j,i} + 1$ for $1 \le j \le m$, then $s_{j,i+1} = k_{j,i+1} + 1$ for $1 \le j \le m$.

$s_{j,1} = k_{j,1} + 1$ for $1 \le j \le m$.

If $s_{j,i} = k_{j,i} + 1$ for $1 \le j \le m$, then $s_{j,i+1} = k_{j,i+1} + 1$.

$\therefore s_{j,i} = k_{j,i} + 1$ for $1 \le j \le m$ and $1 \le i \le n$.

$\therefore t_i = u_i$ for $1 \le i \le n$.

Therefore, our algorithm minimizes $1/n{\cdot}\sum_{i=1}^{n} t_i$. ∎

Another strength of the proposed algorithm is that it guarantees first-come, first-finished. It is not necessary for all inputs in a batch to be created simultaneously. See Fig. 4 again. Since preprocess stops during batch inference, the collected sensor data accumulates if the period of data collection is less than the batch inference latency. These sequential data can then be aggregated to form a batch input. Even when the inputs within a batch are processed on different accelerators, the proposed algorithm ensures that the input generated first will also be the first to complete processing.

Theorem 2. *If $a < b$, $t_a \le t_b$.*

Proof.

From Proof of Theorem 1, $t_i = u_i$ for $1 \le i \le n$.

Therefore, $t_a = u_a \le u_b = t_b$. ∎

Finally, we will show that the proposed algorithm bounds the turn-around time to a constant value.

Theorem 3. *The maximum t_i is $(n + m - 1)/\sum_{j=1}^{m} 1/c_j$.*

Proof.

Let τ_n be scheduled to q_α by our algorithm.

We will derive the inequality of $k_{\alpha,n}$ and $k_{j,n}$ for $1 \le j \le m$ and $j \ne \alpha$.

Since our algorithm schedules q_i to the minimum t_i candidate,

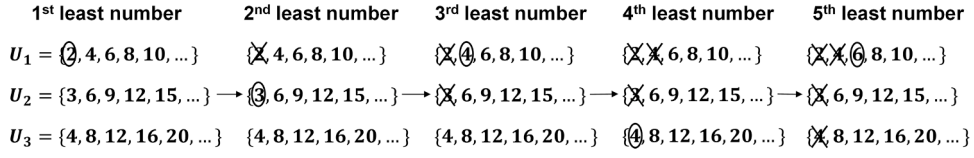


Fig. 11. Simple example for finding the nth least number from sets U_1, U_2, and U_3.

following inequality holds.

$$k_{\alpha,n}\cdot c_\alpha + c_\alpha \le k_{j,n}\cdot c_j + c_j$$

$$\left(k_{\alpha,n}+1\right)\cdot\frac{c_\alpha}{c_j} \le k_{j,n}+1$$

$$\left(k_{\alpha,n}+1\right)\cdot\frac{c_\alpha}{c_j} - 1 \le k_{j,n}$$

We then derive the inequality of t_n.

$$\begin{aligned} n-1 &= \sum_{j=1}^{m} k_{j,n} = \sum_{j=1}^{\alpha-1} k_{j,n} + k_{\alpha,n} + \sum_{j=\alpha+1}^{m} k_{j,n} \\ &\ge \sum_{j=1}^{\alpha-1}\left(\left(k_{\alpha,n}+1\right)\cdot\frac{c_\alpha}{c_j} - 1\right) + k_{\alpha,n} + \sum_{j=\alpha+1}^{m}\left(\left(k_{\alpha,n}+1\right)\cdot\frac{c_\alpha}{c_j} - 1\right) \\ &= \sum_{j=1}^{\alpha-1}\left(\left(k_{\alpha,n}+1\right)\cdot\frac{c_\alpha}{c_j} - 1\right) + \sum_{j=\alpha}^{\alpha}\left(\left(k_{\alpha,n}+1\right)\cdot\frac{c_\alpha}{c_j} - 1\right) + \sum_{j=\alpha+1}^{m}\left(\left(k_{\alpha,n}+1\right)\cdot\frac{c_\alpha}{c_j} - 1\right) \\ &= \sum_{j=1}^{m}\left(\left(k_{\alpha,n}+1\right)\cdot\frac{c_\alpha}{c_j}\right) - m \end{aligned}$$

$$n+m-1 \ge \sum_{j=1}^{m}\left(k_{\alpha,n}+1\right)\cdot\frac{c_\alpha}{c_j}$$

$$\therefore t_n = \left(k_{\alpha,n}+1\right)\cdot c_\alpha \le \frac{n+m-1}{\sum_{j=1}^{m}\frac{1}{c_j}}$$

As same above, we can derive the inequality of t_{n-1}.

$$t_{n-1} \le \frac{n+m-2}{\sum_{j=1}^{m}\frac{1}{c_j}} < \frac{n+m-1}{\sum_{j=1}^{m}\frac{1}{c_j}}$$

Therefore, the maximum t_i is $(n+m-1)/\sum_{j=1}^{m} 1/c_j$.

The maximum t_i is available when τ_n is scheduled to q_α and $k_{\alpha,n}\cdot c_\alpha + c_\alpha = k_{j,n}\cdot c_j + c_j$ for $1 \le j \le m$ and $j \ne \alpha$. ∎

4.3. Postprocessing parameter adjuster

The postprocessing parameter adjuster must determine when and which parameters to provide in order to minimize the arithmetic mean of p_i while maintaining the minimum accuracy. In the target hardware, the LiteRT model representation utilized by the GPU and DSP exhibits lower accuracy than the proprietary model representations used by the Mobilint and Hailo NPUs. Consequently, when o_i has been executed on the Mobilint and Hailo NPUs, the postprocessing parameter adjuster provides parameters that reduce p_i by compromising a_i.

To determine the specific parameters to adjust, it is necessary to define the scope of postprocess. This paper focuses on Non-Maximum Suppression (NMS) postprocess [23]. This postprocessing technique has been extensively used in various computer vision tasks, including object detection and motion estimation, to enhance the accuracy of the results. The NMS postprocess identifies the most relevant bounding boxes among a multitude of bounding boxes.

To identify adjustable parameters, we have evaluated the impact of three representative NMS parameters on accuracy and postprocessing latency. The parameters of interest include the IoU threshold, the score threshold, and the maximum detection number. The IoU threshold determines the maximum overlap permitted between bounding boxes prior to the suppression of one. The score threshold serves to filter out bounding boxes based on their confidence scores during suppression. The maximum detection number limits the number of detections selected after suppression.

The evaluation revealed that all three parameters significantly influence accuracy. However, only increasing the score threshold led to a reduction in postprocessing latency. Therefore, we decide to provide a higher score threshold than the default value to reduce p_i when o_i has been executed on the Mobilint and Hailo NPUs.

The postprocessing parameter adjuster works in three steps. First, the developer must provide the set of tuples of score threshold and mAP for each accelerator at pre-runtime. Second, at runtime, after model execution is complete, the postprocessing parameter adjuster receives information from the accelerator scheduler about which accelerator will generate each output. Finally, the postprocessing parameter adjuster finds the optimal score threshold for o_i that has been executed on the Mobilint and Hailo NPUs. The score threshold is increased such that a_i from Mobilint and Hailo NPUs will be not less than the accuracy from the DSP and GPU.

5. Experimental evaluation

We evaluated the effectiveness of our proposed inference framework against both the proprietary inference frameworks provided by the NPU vendors and the open-source framework LiteRT. For this evaluation, we implemented a test application, which will be described in detail. Subsequently, we will present the experimental setup and results.

5.1. Test application

We have developed an object detection application that takes a COCO 2017 validation dataset [24] comprising 5000 images as input. The application is illustrated in Fig. 12. Initially, the application selects n images from the 5000 images. The preprocessing threads then preprocess the images to generate a batch input. Subsequently, the inference thread initiates batch inference via the LiteRT's batch inference API. As the output is generated, the postprocessing threads perform the postprocess. The application iterates until all 5000 images are detected.

In this application, we restrict the preprocessing and postprocessing threads to run on four of the eight CPU cores in the target system. We then constrain the inference thread to run on the other four cores. The inference thread invokes the accelerator scheduler, which forks worker threads for the accelerators (see Fig. 7). As a result, the inference thread and the worker threads run on different cores than the preprocessing and postprocessing threads. This prevents inference from interfering with preprocess and postprocess. We use the `pthread_setaffinity_np()` Linux function to set the thread affinity.

For each batch inference, the application measures the sum of the turnaround and postprocessing time for n images. Since the application is iterated over 5000 images, the measurement is iterated $5000/n$ times. The application then calculates the mean of the sum of the turnaround and postprocessing time by dividing the sum of the measurement by

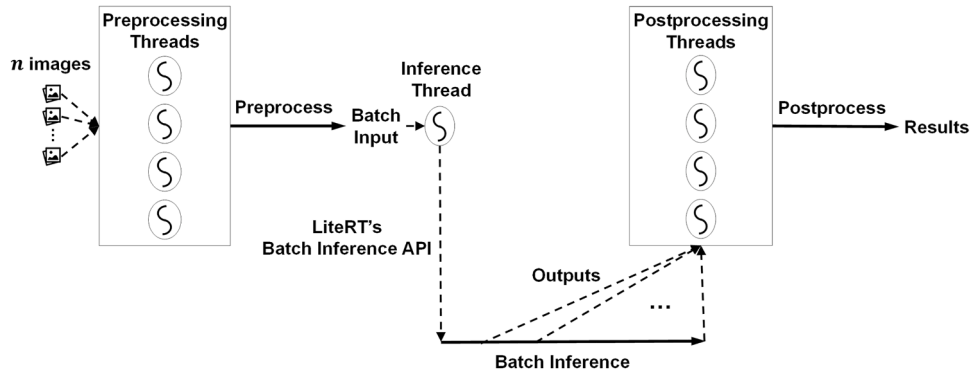


Fig. 12. Test application implemented for experimental evaluation.

5000. Additionally, the maximum value of the sum of the turnaround and postprocessing time is identified. Upon application termination, the total application latency and the mAP are measured. It is important to note that measuring the turnaround time and the postprocessing time separately is not feasible. This limitation arises from the fact that some postprocess is executed within the LiteRT and NPU libraries along with inference, which lack an API to measure the inference and post-processing time.

The application is available in three variants: one written with the LiteRT C++ API, another with the Maccel API, and the third with the HailoRT API. The libjpeg library has been employed for image decoding while the json-c library has been used to output detected objects to a JSON file.

5.2. Experimental setup

Hardware. As previously stated, the target board contains the Qualcomm SA8195 SoC as its primary SoC. This SoC comprises eight CPU cores, a GPU, and a DSP. Two variants of the boards exist. One variant is equipped with a Mobilint Aries NPU connected to the SoC via PCIe, while the other is equipped with three Hailo-8 NPUs.

DL model. We have tried to convert well-known models for the accelerators. The models that have been successfully converted to support multiple accelerators are enumerated in Table 2. The following models were used in the experimental evaluation: SSD Mobilenet v1, v2, EfficientDet Lite 0, 1, 2, YOLO v5, v8, v11 nano, small, medium, large, extra-large, and YOLOv9 compact [25–32]. The compatibility of the DL model with each accelerator is described in Table 2.

Parameters. Two types of parameters have been configured according to the objectives of the experiments. The first is the number of inputs in a batch, denoted as n. The second is the time needed to execute a model on each accelerator, previously denoted as c_j. The time needed to execute a model on a GPU, DSP, Mobilint NPU, Hailo NPU 1, Hailo NPU 2, and Hailo NPU 3 is denoted as c_g, c_d, c_m, c_{h1}, c_{h2}, and c_{h3} respectively.

Table 2
Compatibility of DL model with each accelerator.

Model	GPU	DSP	Mobilint NPU	Hailo NPU
SSD Mobilenet v1, v2	O	O	X	O
EfficientDet Lite 0, 1, 2	O	X	X	O
YOLO v5 nano, small, medium, large, extra-large	X	O	O	X
YOLO v8 nano, small, medium, large, extra-large	X	O	O	O
YOLO v9 compact	X	O	O	O
YOLO v11 nano, small, medium, large, extra-large	X	O	X	O

5.3. Experimental result

In order to assess the effectiveness of the proposed inference framework, we first conducted a comparison of the mean of the sum of the turnaround and postprocessing time, the total application latency, and the mAP. The following configurations were selected as baseline comparisons: LiteRT(GPU), LiteRT(DSP), Maccel, and HailoRT. For the LiteRT(GPU) configuration, we ran the LiteRT application on the original LiteRT with the GPU delegate. For the LiteRT(DSP) configuration, we ran the LiteRT application with the Hexagon delegate. Additionally, the Maccel application was deployed on the board with a Mobilint NPU for the Maccel configuration. Finally, the HailoRT application was executed on the board with the Hailo NPUs for the HailoRT configuration. Only one Hailo NPU was utilized on the HailoRT configuration.

Subsequently, we ran the LiteRT application with the proposed inference framework on the board equipped with the Mobilint NPU and the board equipped with the Hailo NPUs, respectively. We set n to 40 and c_g, c_d, c_m, c_{h1}, c_{h2}, and c_{h3} to the values measured at pre-runtime.

Fig. 13 depicts the result on the board equipped with the Mobilint NPU. The x-axis represents the model, while the y-axis denotes the normalized value for each model and mAP. It should be noted that no model supports both the GPU and the Mobilint NPU (see Table 2). Therefore, our inference framework can only utilize the DSP and the Mobilint NPU concurrently. The proposed inference framework invariably diminished the mean of the sum of the turnaround and post-processing time and the total application latency when compared to the Maccel configuration. The most effective model was the YOLO v5 nano, which exhibited a 33 % reduction in the mean of the sum of the turnaround and postprocessing time and a 16 % reduction in the total application latency. The mAP was not decreased below the value when using the LiteRT(DSP).

The results obtained from the board equipped with the Hailo NPUs are presented in Fig. 14. A blank spot in the graph indicates that the corresponding accelerator is not supported for that specific model. The inference framework concurrently leverages the GPU, DSP, and three Hailo NPUs for the SSD Mobilenet v1 and v2 models. For the EfficientDet Lite 0, 1, 2 models, the GPU and the three Hailo NPUs are utilized. In the case of the YOLO series, the DSP and the three Hailo NPUs are employed. The proposed inference framework consistently demonstrated a reduction in both the mean of the sum of the turnaround and postprocessing time and the total application latency when compared to the HailoRT configuration. The mAP was not reduced below the values observed on the LiteRT(GPU) and LiteRT(DSP) configurations.

The SSD Mobilenet v1 and v2 models were predicted to demonstrate the most significant reduction since these models can run on the greatest number of accelerators in parallel. However, the EfficientDet Lite 0 model demonstrated the most efficacy, achieving 95 % reduction in the mean of the sum of the turnaround and postprocessing time and 91 % reduction in the total application latency. This enhancement can be

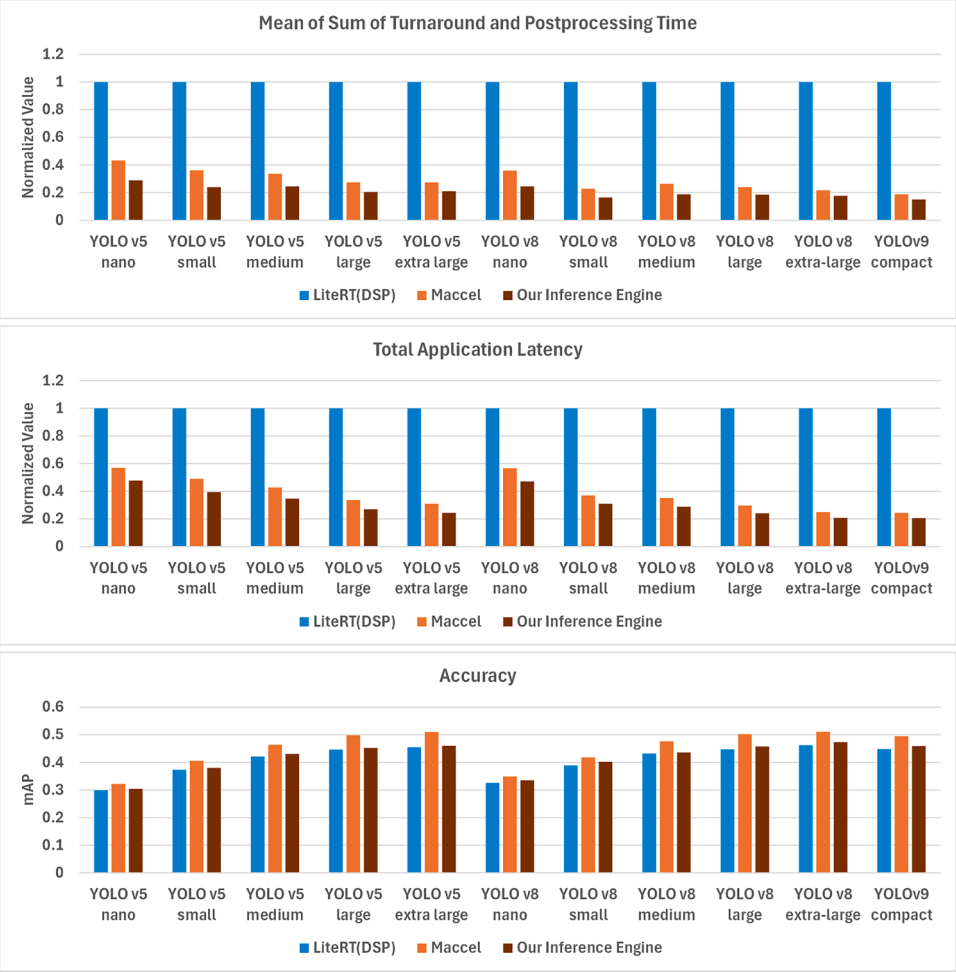


Fig. 13. Comparison of the mean of the sum of the turnaround and postprocessing time, the total application latency, and the mAP on the board equipped with the Mobilint NPU.

attributed to the fact that the postprocess for the EfficientDet Lite models requires more time than that for the other models. Empirical evidence supported this finding, demonstrating that the mean of the sum of the turnaround and postprocessing time drastically decreased when increasing the score threshold while running the EfficientDet Lite models on the HailoRT configuration. Since the proposed inference framework reduces the postprocessing latency, the most efficacy was observed for the EfficientDet Lite models.

Among the non-EfficientDet models, as expected, the SSD Mobilenet v2 exhibited the most reduction in both the mean of the sum of the turnaround and postprocessing time and the total application latency, with a 72 % and 41 % decrease, respectively.

To assess the functionality of the accelerator scheduler, we focused on SSD Mobilenet v2, which can run on the greatest number of accelerators in parallel. We changed n from 1 to 20 and ran the LiteRT application with the proposed inference framework on the board equipped with the Hailo NPUs. The ratio of c_{h1}, c_{h2}, c_{h3}, c_d, and c_g that were measured at pre-runtime was 1.00 : 1.00 : 1.00 : 0.971 : 2.71. Therefore, as n increased from 1 to 8, the input was allocated to the DSP, the Hailo NPU 1, the Hailo NPU 2, and the Hailo NPU 3 successively. Notably, when n reached 9, the input was assigned to the GPU. When n increased from 10 to 20, the input was assigned to the DSP, the Hailo NPU 1, the Hailo NPU 2, and the Hailo NPU 3 in turn.

Fig. 15 depicts the maximum value of the sum of the turnaround and postprocessing time according to the batch size n. This result provides empirical evidence to support the functionality of the accelerator scheduler. As the batch size increases, the maximum value of the sum of turnaround and postprocessing time increases stepwise, with a step size of 4, which corresponds to the number of DSP and Hailo NPUs. However, an intermediate value was observed when the batch size was 9, indicating that the input was assigned to the GPU.

Although the scheduler has functioned as anticipated, the performance of three Hailo NPUs in Fig. 14 did not align with our expectations, prompting an investigation to understand the underlying causes. To this end, we measured the mean of the sum of the turnaround and postprocessing time of YOLO v8 series when using one Hailo NPU only, two Hailo NPUs, and three Hailo NPUs. To execute a single Hailo NPU, we configured c_{h1} to 1 and c_{h2}, c_{h3}, c_d, and c_g to infinite value. To assess the impact of two Hailo NPUs, we set c_{h1} and c_{h2} to 1 and c_{h3}, c_d, and c_g to infinite value. Finally, to ascertain the impact of three Hailo NPUs, we set c_{h1}, c_{h2}, and c_{h3} to 1 and c_d and c_g to infinite value. The parameter n was set to 40.

As demonstrated in Fig. 16, the mean of the sum of the turnaround and postprocessing time on the two Hailo NPUs was between 52 % and 70 % of that on the single Hailo NPU. The time on the three Hailo NPUs was between 38 % and 64 % of the value on the single Hailo NPU. While the utilization of three Hailo NPUs led to a substantial reduction in time compared to the utilization of two Hailo NPUs on the YOLO v8 nano and small models, the reduction was minimal on the YOLO v8 medium, large, and extra-large models.

The underlying reason for this phenomenon is likely insufficient PCIe bandwidth, given that the SA8195 SoC only supports 1 × 4-lane PCIe 3.0. The three Hailo-8 NPUs have to share this PCIe bandwidth;

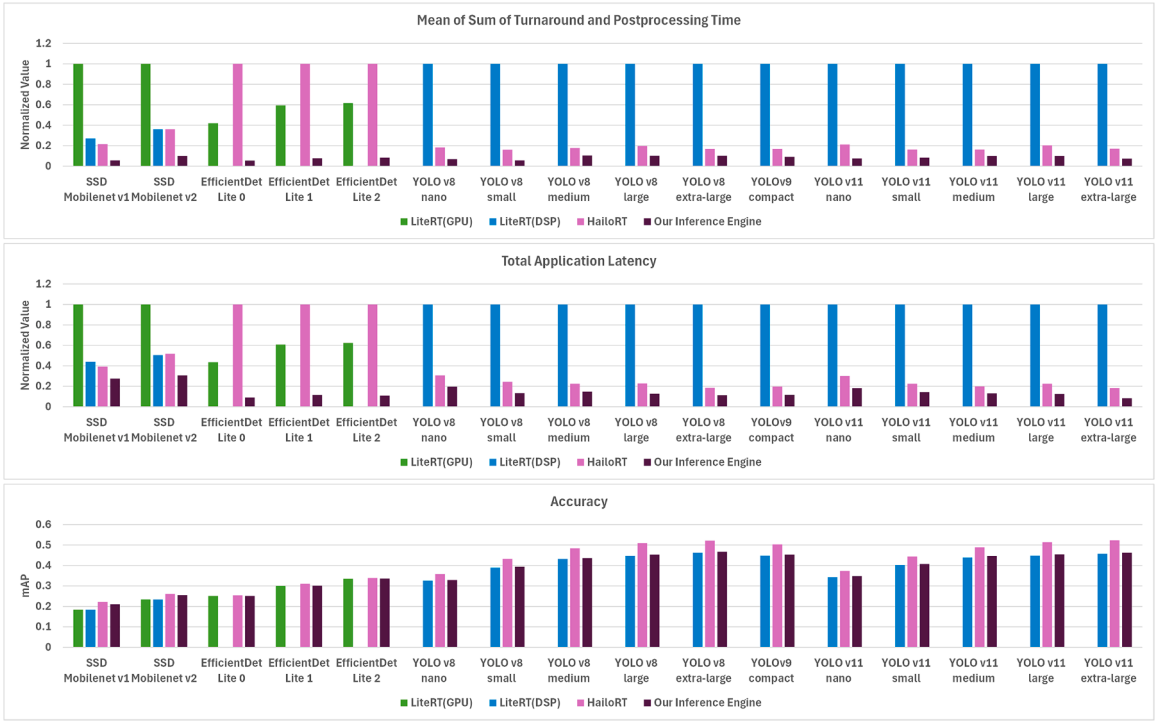


Fig. 14. Comparison of the mean of the sum of the turnaround and postprocessing time, the total application latency, and the mAP on the board equipped with the Hailo NPUs.

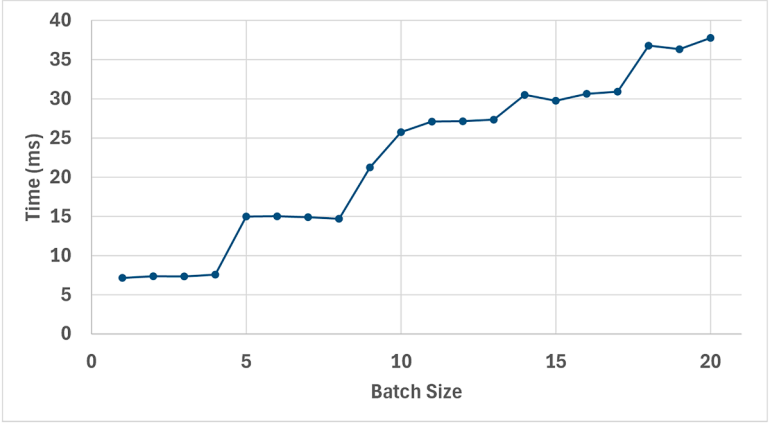


Fig. 15. The maximum value of the sum of the turnaround and postprocessing time according to the batch size.

however, each NPU may require at least 2-lane PCIe 3.0 connection to accommodate large models. This finding is supported by the specification sheet [33].

Finally, we compared the maximum value of the sum of the turnaround and postprocessing time to the theoretical value in Theorem 3. We ran the LiteRT application with the proposed inference framework on the board equipped with the Mobilint NPU and the board equipped with the Hailo NPUs, respectively. We set n to 40 and c_g, c_d, c_m, and c_{h1} to the values measured at pre-runtime. c_{h2} and c_{h3} were set to infinite value considering the PCIe bandwidth interference between multiple Hailo NPUs.

Fig. 17 depicts the result. The value observed on the proposed inference framework was always smaller than the theoretical value.

6. Related works

DL models are typically trained on high-performance servers equipped with powerful CPUs and GPUs. To facilitate the deployment of DL models on resource-constrained embedded systems, a substantial amount of research has focused on optimizing the DL inference. Early research concentrated on enhancing inference speed on a single accelerator [25,34–44]. Subsequent research has progressed to speed up inference on homogeneous accelerators [45–50] and then on heterogeneous accelerators [51,52].

Single accelerator. Improving inference speed on a single accelerator involves optimizing the DL model itself. The optimization can be categorized into three approaches: (1) modifying the overall model structure, (2) altering the layers within the model, and (3) adjusting the nodes within individual layers. Modifying the model structure by adjusting the number of depths and widths of the model to generate models of varying sizes is known as model scaling [25,37,38]. EfficientNets, for example, utilize model scaling to generate models ranging from 7.8 MB to 66 MB, achieving accuracy between 79 % and 84 %.

Modifying the layers within the model involves either removing

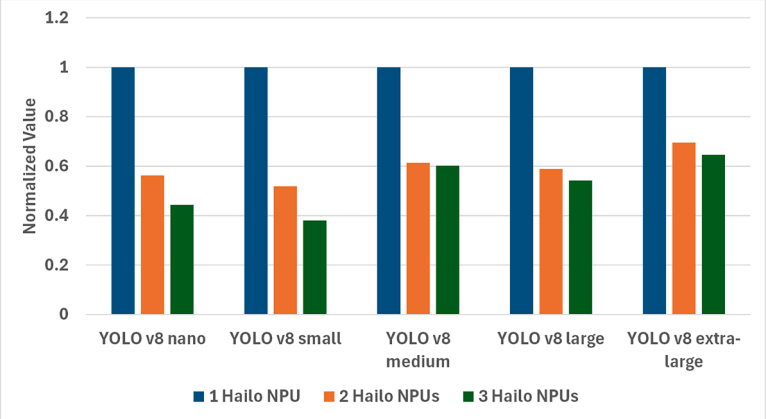


Fig. 16. Mean of the sum of the turnaround and postprocessing time on Hailo NPUs.

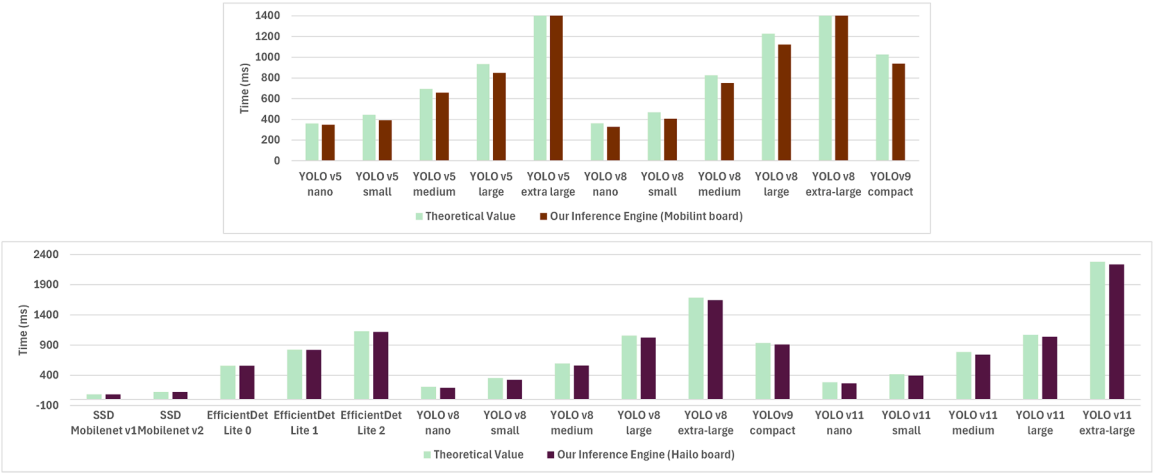


Fig. 17. Comparison of the maximum of the sum of the turnaround and postprocessing time to the theoretical value.

redundant weights or compressing weights by reducing their precision. The former is referred to as pruning [39–41] and the latter is called quantization [34–36]. Adjusting the nodes within individual layers aims to design models that are hardware-friendly for specific devices. This often involves replacing operators or parameters within a node with those that execute efficiently on the target hardware [42–44]. It is important to note that these model optimization techniques are complementary to accelerating models with multiple accelerators and have been employed in subsequent research.

Homogeneous accelerators. To speed up inference on homogeneous accelerators, researchers have proposed strategies for dynamically dividing a model into subgraphs and assigning these subgraphs at runtime to multiple GPUs or NPUs using parallel or pipelined execution. Young H. Oh and Kiwon Sohn proposed partitioning and scheduling subgraphs to maximize the utilization of multi-core NPUs [47,48]. Yunyi Xiang and Arne Symons proposed genetic algorithm-based schedulers for multi-core accelerators to reduce application latency [49,50]. Rafael Sousa's compiler-assisted scheduler employed tensor slicing to minimize data transfers between the host and the NPU cores' memories [45]. Based on experimental heuristics, Hanwoong Jung proposed partitioning and scheduling algorithms for multi-core NPUs, considering synchronization across multiple cores and communication between local and global memory [46].

Heterogeneous accelerators. The emergence of heterogeneous accelerators has spurred research into partitioning models for concurrent execution across these heterogeneous resources. Leading DL inference frameworks like LiteRT, ONNX Runtime, and ExecuTorch are designed with the goal of supporting such heterogeneous execution. They incorporate internal mechanisms to partition models into subgraphs, potentially assigning each to a different accelerator—utilizing Execution Providers (EPs) in ONNX Runtime, backends in ExecuTorch, and delegates in LiteRT.

However, despite these design intent, practical limitations within the respective backend libraries often constrain execution to a single accelerator at a time. For instance, in ONNX Runtime, specifying multiple EPs should ideally enable model partitioning and concurrent execution, but this capability is frequently unsupported by individual EPs. Many EPs, including those for Intel OpenVINO, Qualcomm QNN, Android NNAPI, and Apple Core ML, treat ONNX graph as merely a model description format, converting the entire graph into their proprietary intermediate representation. These EPs implement the GetCapability() function primarily to check if the provided graph is a subgraph (using IsSubgraph()) and reject processing if it is, thereby preventing model partitioning. Even EPs that support subgraph execution, like those for NVIDIA and AMD GPUs, do not currently enable ONNX Runtime to execute a single model concurrently across multiple GPUs, a known limitation with pending feature requests [53].

ExecuTorch and LiteRT face analogous challenges. Although ExecuTorch theoretically allows specifying multiple backends, most backends require Ahead-Of-Time (AOT) compilation via the to_edge_transform_and_lower() function. This process converts the model into a target-specific representation—a step required for backends targeting Apple Core ML, Qualcomm QNN, and Arm Ethos-U NPUs. Once a model is lowered for a specific backend, it cannot be dynamically

retargeted to run parts on another accelerator. Similarly, LiteRT faces restrictions regarding the concurrent application of multiple delegates; currently, only the CPU XNNPACK delegate is designed for cooperative execution alongside others. Notably, LiteRT delegates for GPUs and Hexagon DSPs do not mandate conversion to a proprietary intermediate representation. This suggests a potential pathway for modifying LiteRT to enable concurrent execution of a single model across these specific delegates. Indeed, prior works by Joo Seong Jeong and Jianyu Wei leveraged this characteristic, using the LiteRT model as a common representation to run partitioned models in parallel on CPUs, GPUs, and DSPs via their respective delegates [51,52]. Nonetheless, this strategy remains inapplicable to delegates reliant on proprietary representation conversion.

Positioning of our work. In summary, existing frameworks face significant limitations, primarily stemming from backend library restrictions that impede concurrent execution across heterogeneous accelerators. Compounding this issue, they typically employ simplistic greedy scheduling strategies. For example, ONNX Runtime, ExecuTorch, and LiteRT attempt to offload as much computation as possible to the first specified (or highest priority) accelerator. This heuristic approach does not guarantee minimal turnaround time. Moreover, even if backend library limitations were resolved to allow static partitioning and offline conversion for multiple hardware targets, this approach would still lack the flexibility to adapt scheduling decisions dynamically at runtime.

In contrast, our proposed inference engine overcomes these limitations. It circumvents backend library restrictions by leveraging the batch inference API, thereby enabling concurrent execution even with backends not inherently designed for cooperation. Critically, departing from simplistic greedy approach, our framework employs an optimal scheduling strategy to dynamically distribute computation across heterogeneous accelerators at runtime. This dynamic distribution effectively minimizes turnaround time, offering a more performant and adaptable solution compared to current state-of-the-art frameworks.

7. Conclusion

This paper presented a novel inference framework designed to accelerate deep learning inference on heterogeneous accelerators. By leveraging a batch inference API and incorporating a portable hooking engine for commercial NPUs, our framework enables concurrent processing of multiple inputs across heterogeneous accelerators via LiteRT. The proposed scheduling algorithm demonstrably minimizes average turnaround time. Furthermore, our adaptive postprocessing parameter adjuster minimizes postprocessing latency while maintaining the minimum accuracy. Evaluation on a Qualcomm SA8195 SoC with Mobilint and Hailo NPUs demonstrated a significant reduction in application latency compared to existing inference frameworks. Future work will focus on developing scheduling algorithms for scenarios involving concurrent execution of multiple DL models across accelerators, with a particular emphasis on optimizing energy efficiency.

CRediT authorship contribution statement

Philkyue Shin: Visualization, Supervision, Resources, Methodology, Formal analysis, Conceptualization, Writing – original draft, Validation, Software, Project administration, Investigation, Data curation. **Myungsun Kim:** Writing – review & editing. **Seongsoo Hong:** Writing – review & editing, Funding acquisition.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Seongsoo Hong reports financial support was provided by Korea Ministry of Trade Industry and Energy. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the Technology Innovation Program (20014336, Autonomous Driving AI SW Platform & Tool Chain based on Standard Architecture) funded By the Ministry of Trade Industry & Energy (MOTIE, Korea).

Data availability

Data will be made available on request.

References

[1] Sauptik Dhar, et al., A survey of on-device machine learning: an algorithms and learning theory perspective, ACM Trans. Internet of Things 2 (3) (2021) 1–49.
[2] Sicong Liu, et al., Deep learning inference on heterogeneous mobile processors: potentials and pitfalls, in: Proceedings of the 2024 Workshop on Adaptive AIoT Systems, 2024.
[3] Runhua Zhang, et al., A high-performance dataflow-centric optimization framework for deep learning inference on the edge, J. Syst. Archit. 152 (2024) 103180.
[4] Abdallah Moujahid, et al., Machine learning techniques in ADAS: a review, in: 2018 International Conference on Advances in Computing and Communication Engineering (ICACCE), 2018.
[5] Miaomiao Liu, Sikai Yang, Wyssanie Chomsin, Wan Du, Real-time tracking of smartwatch orientation and location by multitask learning, in: Proceedings of SenSys 120–133, 2022.
[6] Junchao Yang, et al., Joint optimization of layering and power allocation for scalable VR video in 6G networks based on Deep Reinforcement Learning, J. Syst. Archit. (2025) 103401.
[7] Daniel Caballero-Martin, et al., Artificial intelligence applied to drone control: a state of the art, Drones 8 (7) (2024) 296.
[8] https://developer.nvidia.com/deep-learning-accelerator. Accessed 25 Feb. 2025.
[9] https://docs.nvidia.com/vpi/architecture.html. Accessed 25 Feb. 2025.
[10] https://docs.qualcomm.com/bundle/publicresource/topics/80-63442-50/backend.html. Accessed 25 Feb. 2025.
[11] Peng Du, et al., From CUDA to OpenCL: towards a performance-portable solution for multi-platform GPU programming, Parallel. Comput. 38 (8) (2012) 391–407.
[12] https://www.qualcomm.com/developer/software/hexagon-npu-sdk. Accessed 25 Feb. 2025.
[13] https://www.mobilint.com/sdk. Accessed 25 Feb. 2025.
[14] https://github.com/hailo-ai/hailort. Accessed 25 Feb. 2025.
[15] https://developer.android.com/ndk/guides/neuralnetworks. Accessed 25 Feb. 2025.
[16] https://ai.google.dev/edge/litert. Accessed 25 Feb. 2025.
[17] https://onnxruntime.ai. Accessed 25 Feb. 2025.
[18] https://pytorch.org/executorch-overview. Accessed 25 Feb. 2025.
[19] Jihun Kim, Dong Seog Han, Benaoumeur Senouci, Radar and vision sensor fusion for object detection in autonomous vehicle surroundings, in: 2018 tenth international conference on ubiquitous and future networks (ICUFN), 2018.
[20] https://www.katech.re.kr/eng. Accessed 25 Feb. 2025.
[21] https://github.com/Hozzu/TFLite-for-Heterogeneous-Accelerators. Accessed 24 April. 2025.
[22] British Standards Institute: The C++ Standard incorporat ing Technical Corrigendum 1. John Wiley and Sons, Ltd., 2003. ISBN 0-470-84674-7.
[23] Jan Hosang, Rodrigo Benenson, Bernt Schiele, Learning non-maximum suppression, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2017.
[24] https://cocodataset.org. Accessed 25 Feb. 2025.
[25] Howard, Andrew G. "Mobilenets: Efficient convolutional neural networks for mobile vision applications." *arXiv preprint* arXiv:1704.04861. 2017.
[26] Wei Liu, et al., Ssd: single shot multibox detector, in: Computer Vision–ECCV2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part I 14, 2016.
[27] Yu-Chen Chiu, et al., Mobilenet-SSDv2: an improved object detection model for embedded systems, in: 2020 International conference on system science and engineering (ICSSE), 2020.
[28] Mingxing Tan, Ruoming Pang, Quoc V. Le, Efficientdet: scalable and efficient object detection, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2020.
[29] Joseph Redmon, et al., You only look once: unified, real-time object detection, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2016.
[30] Jocher, Glenn, Ayush Chaurasia, and Jing Qiu. "Ultralytics YOLOv8." https://github.com/ultralytics/ultralytics. 2023.

[31] Chien-Yao Wang, I-Hau Yeh, Hong-Yuan Mark Liao, Yolov9: learning what you want to learn using programmable gradient information, in: European conference on computer vision, Cham: Springer Nature Switzerland, 2024.
[32] Jocher, Glenn and Jing Qiu. "Ultralytics YOLO11." https://github.com/ultralytics/ultralytics. 2024.
[33] https://hailo.ai/products/ai-accelerators/hailo-8-m2-ai-acceleration-module/#hailo8-m2-overview. Accessed 25 Feb. 2025.
[34] Finkelstein, Alexander, Uri Almog, and Mark Grobman. "Fighting quantization bias with bias." *arXiv preprint* arXiv:1906.03193. 2019.
[35] Alex Finkelstein, et al., Qft: post-training quantization via fast joint finetuning of all degrees of freedom, in: European Conference on Computer Vision, 2022.
[36] Benoit Jacob, et al., Quantization and training of neural networks for efficient integer-arithmetic-only inference, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2018.
[37] Kaiming He, et al., Deep residual learning for image recognition, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2016.
[38] Mingxing Tan, Quoc Le, Efficientnet: rethinking model scaling for convolutional neural networks, in: International conference on machine learning, 2019.
[39] Yang He, et al., Filter pruning via geometric median for deep convolutional neural networks acceleration, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2019.
[40] Yihui He, Xiangyu Zhang, Jian Sun, Channel pruning for accelerating very deep neural networks, in: Proceedings of the IEEE international conference on computer vision, 2017.
[41] Tien-Ju Yang, Yu-Hsin Chen, Vivienne Sze, Designing energy-efficient convolutional neural networks using energy-aware pruning, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2017.
[42] Xiaoliang Dai, et al., Chamnet: towards efficient network design through platform-aware model adaptation, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019.
[43] Mingxing Tan, et al., Mnasnet: platform-aware neural architecture search for mobile, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2019.
[44] Tien-Ju Yang, et al., Netadapt: platform-aware neural network adaptation for mobile applications, in: Proceedings of the European conference on computer vision (ECCV), 2018.
[45] Rafael Sousa, et al., Tensor slicing and optimization for multicore NPUs, J. Parallel. Distrib. Comput. 175 (2023) 66–79.
[46] Hanwoong Jung, et al., Accelerating deep neural networks on mobile multicore NPUs, in: Proceedings of the 21st ACM/IEEE International Symposium on Code Generation and Optimization, 2023.
[47] Young H. Oh, et al., Layerweaver: maximizing resource utilization of neural processing units via layer-wise scheduling, in: 2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2021.
[48] Kiwon Sohn, et al., A strategy to maximize the utilization of AI neural processors on an automotive computing platform, in: 2024 IEEE International Conference on Consumer Electronics (ICCE), 2024.
[49] Yunyi Xiang, et al., Aries: a DNN inference scheduling framework for multi-core accelerators, in: Proceedings of the 2024 5th International Conference on Computing, Networks and Internet of Things, 2024.
[50] Arne Symons, et al., Stream: a modeling framework for fine-grained layer fusion on multi-core DNN accelerators, in: 2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), 2023.
[51] Joo Seong Jeong, et al., Band: coordinated multi-dnn inference on heterogeneous mobile processors, in: Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services, 2022.
[52] Jianyu Wei, et al., Nn-stretch: automatic neural network branching for parallel inference on heterogeneous multi-processors, in: Proceedings of the 21st Annual International Conference on Mobile Systems, Applications and Services, 2023.
[53] https://github.com/microsoft/onnxruntime/issues/16382. Accessed 24 April. 2025.



Philkyue Shin earned his B.S. degrees in the Department of Electrical and Computer Engineering from Seoul National University, Korea, in 2017. He is currently a Ph.D. candidate in the Department of Electrical and Computer Engineering at Seoul National University, Korea. He is also a member of Real-Time Operating System Laboratory at Seoul National University. His current research interests include task scheduling and resource management in the Linux kernel and software architecture for embedded AI.



Myungsun Kim is currently an Associate Professor with the Department of Applied Artificial Intelligence, Hansung University, Seoul, South Korea. Prior to joining the university, he had worked in Samsung Electronics R&D for the development of NPU (Neural Processing Unit), Linux kernel internal, Android framework, and SoC. He received the Ph.D degree from the Department of Electrical and Computer Engineering, Seoul National University, in 2016, and the B.S and M.S degrees in Electrical Engineering from Chung-Ang University, in 1998 and 2000, respectively. His current research interests include generative AI, embedded system design, HW/SW cross-layer optimization for on-device AI.



Seongsoo Hong earned his B.S. and M.S. degrees in computer engineering from Seoul National University, Korea, in 1986 and 1988, respectively. He received his Ph.D. degree in computer science from the University of Maryland, College Park, in 1994. He is currently a professor in the Department of Electrical and Computer Engineering at Seoul National University. His current research interests include software-defined vehicles (SDV) and cloud native computing, task scheduling and resource management in the Linux kernel, embedded and real-time systems design, real-time operating systems, software architecture for embedded and real-time systems, and cross-layer optimization of complex multi-layered software systems. He is the member of the advisory committee of IEEE RTCSA. He served as a general co-chair of IEEE RTCSA 2006 and CASES 2006 and as a program committee co-chair of IEEE RTAS 2005, RTCSA 2003, IEEE ISORC 2002 and ACM LCTES 2001. He has served on numerous program committees, including IEEE RTSS and ACM OOPSLA. He is a fellow of the National Academy of Engineering of Korea (NAEK). He is currently a senior member of the IEEE and a senior member of the ACM.